The Self-Pruning Neural Network: Learning Sparse Architectures During Training

Author: Research Implementation Report
Dataset: CIFAR-10 | **Framework:** PyTorch | **Hardware:** NVIDIA T4 GPU (Google Colab)

Abstract

Modern deep neural networks achieve state-of-the-art performance at the cost of significant parameter redundancy, making deployment on resource-constrained hardware a persistent challenge. This work presents a self-pruning architecture in which each weight is paired with a learnable gate parameter, enabling the network to discover its own sparse structure during end-to-end training rather than through post-hoc pruning. Gates are regularized via an L1 penalty on their sigmoid-activated values, which induces a constant downward pressure that drives redundant gates to exactly zero. We evaluate the approach on CIFAR-10 across three regularization strengths ($\lambda \in \{1e\text{-}5, 1e\text{-}4, 1e\text{-}3\}$), demonstrating that a well-calibrated λ can eliminate over 91% of network connections while *improving* test accuracy relative to lighter regularization. A consistent delayed pruning phenomenon — where sparsity remains at 0% for the first ~19 epochs before collapsing rapidly — is identified and analyzed as a signature of the gating mechanism's learning dynamics.

---

1. Introduction

1.1 The Cost of Overparameterization

The empirical success of deep learning has come hand-in-hand with a relentless growth in model size. Models with hundreds of millions of parameters have become standard, yet research consistently shows that large fractions of those parameters contribute negligibly to predictive performance. This redundancy has tangible costs: increased memory footprint, higher inference latency, elevated energy consumption, and practical barriers to deployment on edge devices.

Network pruning — the selective removal of weights, neurons, or entire layers — has emerged as one of the most effective techniques for addressing these costs. Classical pruning pipelines, however, are inherently sequential: (1) train a full dense model, (2) identify and remove unimportant weights using a magnitude or gradient-based criterion, (3) fine-tune to recover lost accuracy. This three-stage cycle is computationally wasteful, operationally

cumbersome, and conceptually unsatisfying: the network spends the majority of its training budget learning connections it will subsequently discard.

1.2 Self-Pruning as a Training Objective

A more elegant formulation is to make sparsity a differentiable objective and optimize it jointly with the classification loss, allowing the network to simultaneously learn *what* to represent and *which connections matter* for that representation. This is the core idea explored here. By augmenting every weight with a learnable gate, and penalizing the sum of active gates during training, we create an end-to-end system that discovers its own sparse architecture — no post-processing required.

This approach sits at the intersection of several active research threads: variational dropout, lottery ticket hypothesis work, and L0 regularization methods. The implementation here is deliberately minimal, prioritizing clarity and empirical interpretability over architectural complexity.

---

2. Methodology

2.1 Prunable Layer Design

The fundamental building block is the `PrunableLinear` layer, a drop-in replacement for `nn.Linear` that augments each weight with a co-learned gate parameter. Formally, for a layer with weight matrix $\mathbf{W} \in \mathbb{R}$^(out × in) and gate matrix $\mathbf{G} \in \mathbb{R}$^(out × in):

$$\mathbf{W}' = \mathbf{W} \odot \sigma(\mathbf{G})$$

$$\text{output} = \mathbf{W}'\mathbf{x} + \mathbf{b}$$

where σ denotes the element-wise sigmoid function and $\odot$ is the Hadamard (element-wise) product. Both **W** and **G** are `nn.Parameter` instances, updated jointly by the optimizer.

Initialization design decisions:

Parameter	Initialization	Rationale
`weight`	Kaiming uniform	Preserves activation variance at initialization, matching `nn.Linear` defaults
`gate_scores`	Constant 0.0	$\sigma(0) = 0.5 \rightarrow$ all gates start at 50% open; a maximally neutral, uncommitted prior
`bias`	Uniform $\pm 1/\sqrt{\text{fan_in}}$	Standard PyTorch convention

The choice to initialize gate scores at exactly 0 is particularly important: it ensures the network begins training with all pathways partially active, allowing gradients to flow freely and giving the optimizer equal opportunity to either open or close each gate.

2.2 Forward Pass

The forward pass is computationally simple and fully differentiable:

```
    def forward(self, x):
        gates         = torch.sigmoid(self.gate_scores)   # squash to
(0, 1)
        pruned_weight = self.weight * gates               # element-
wise gating
        return F.linear(x, pruned_weight, self.bias)
```

Since $\sigma(\cdot)$ is smooth everywhere, gradients propagate through gates without any discontinuity. The effective weight seen by the data is a continuously modulated version of the raw weight — not a hard binary mask. This stands in contrast to straight-through estimator methods, where discontinuous masking requires gradient approximations. The sigmoid gate formulation is fully compatible with standard optimizers (Adam is used here) and requires no special handling.

2.3 Loss Function

The total training loss decomposes as:

$$\mathrm{L}_{total} = \mathrm{L}_{CE} + \lambda \sum_{\ell} \sum_{i,j} G_{ij}^{(\ell)})$$

where the first term is cross-entropy classification loss and the second is the sparsity penalty summed over all `PrunableLinear` layers.

Why L1 and not L2?
An L2 penalty on gate values (i.e., penalizing $\sigma(G)^2$) exerts a force proportional to the gate's current magnitude — as a gate shrinks, the penalty gradient also shrinks, allowing the gate to stabilize at some small positive value rather than reaching zero. An L1 penalty (penalizing $|\sigma(G)| = \sigma(G)$ since gates are non-negative) exerts a **constant downward pressure** independent of the gate's current value. This constant force is precisely what drives gates all the way to zero, producing true sparsity rather than mere shrinkage.

λ as a capacity control knob:
λ governs the tradeoff between task performance and architectural compactness. A small λ barely perturbs the classification objective; a large λ dominates the loss, aggressively suppressing connectivity at the cost of accuracy. Selecting λ is therefore equivalent to selecting a point on the Pareto frontier between accuracy and sparsity.

The sparsity penalty operates on the raw gate sum (not a normalized mean), which is intentional: the quantity we wish to minimize is the total number of active connections, and the sum scales with this quantity directly.

2.4 Training Configuration

Hyperparameter	Value
Dataset	CIFAR-10 (50K train / 10K test)
Architecture	3-layer MLP: 3072 → 1024 → 512 → 10
Optimizer	Adam (lr = 1e-3)
Batch size	128
Epochs	30
Gate threshold	0.01 (gates below this counted as pruned)
λ values tested	1e-5, 1e-4, 1e-3

Data augmentation (training only): random crop with 4px padding, random horizontal flip, per-channel normalization using CIFAR-10 statistics (μ = [0.4914, 0.4822, 0.4465], σ = [0.2023, 0.1994, 0.2010]).

The network architecture is a fully-connected MLP: CIFAR-10 images (3×32×32 = 3,072 features) are flattened and passed through two hidden layers (1,024 and 512 units respectively, with ReLU activations) before the 10-class output layer. All three linear layers

are replaced with `PrunableLinear`. Total learnable parameters: approximately 4.2M weights + 4.2M gate scores = **~8.4M parameters**.

---

3. Key Insight: Delayed Pruning Behavior

One of the most striking and theoretically interesting phenomena observed across all three experimental runs is a consistent pattern of **delayed pruning onset**: sparsity remains exactly 0% for the first 19 epochs, then rises sharply in epoch 20 and continues accelerating.

Epoch-by-epoch trace for λ = 1e-4:

Epoch	Sparsity
1–19	0.0%
20	63.7%
21	75.7%
22	81.0%
30	91.8%

This is not a threshold artifact — the same qualitative pattern holds for all three λ values, with epoch 20 marking the transition in every case.

Mechanistic explanation:
During early training, the Adam optimizer is taking large, exploratory gradient steps to find a useful loss basin. At this stage, the classification signal dominates: the network is actively building representations, and the gate penalty is insufficient to overcome the classification gradient's desire to keep weights active. Gates hover near 0.5 (their initialization) as the optimizer balances both objectives.

As classification loss plateaus — the network has learned most of what it can from the available representation — the marginal cost of closing a gate (lost classification signal) decreases relative to the fixed benefit of closing it (reduced sparsity penalty). This creates a **phase transition**: once the first gates close, the representation adapts, making additional closures cheaper, triggering an avalanche of pruning.

Why this is beneficial:
This "learn-first, prune-second" ordering is functionally superior to pruning from the outset. The network first discovers *which connections are useful* (using the full-capacity dense network as an exploration vehicle), then systematically eliminates those that are not. A network that prunes aggressively from initialization would never develop rich enough representations to make informed pruning decisions. The delayed behavior is therefore not a failure mode — it is evidence that the gating mechanism is working as intended.

---

4. Results & Analysis

4.1 Summary Table

λ	Test Accuracy	Sparsity	Notes
1e-5	58.55%	51.07%	Low pruning pressure; moderate sparsity
1e-4	**59.00%**	**91.82%**	**Best model; high sparsity with peak accuracy**
1e-3	51.45%	99.81%	Over-regularized; capacity collapse

4.2 The Sparsity–Accuracy Tradeoff

The results reveal a non-monotonic relationship between regularization strength and accuracy, with a clear sweet spot at λ = 1e-4:

λ = 1e-5 (under-regularized): The penalty is too weak to enforce meaningful sparsity. Only 51% of gates are pruned — barely more than the random 50% implied by initialization — suggesting the optimizer sees little incentive to commit to pruning. The relatively low accuracy (58.55%) indicates the network may also be overfitting due to insufficient regularization.

λ = 1e-4 (optimal): A 10× increase in penalty pressure yields a dramatic jump to 91.8% sparsity *and* the highest test accuracy (59.00%). This counterintuitive result — that more aggressive pruning improves generalization — is consistent with the **double descent** and regularization literatures: the sparsity constraint acts as a strong implicit regularizer, reducing the effective model capacity to a range that matches the dataset's intrinsic complexity.

λ = 1e-3 (over-regularized): At this strength, the sparsity penalty so thoroughly dominates the loss that the classification objective barely contributes. 99.8% of gates collapse to zero, leaving a network with only ~0.2% of its original connections — insufficient to represent the 10-class distinctions in CIFAR-10. Test accuracy drops to 51.45%, only marginally above random chance for a 10-class problem.

4.3 Insightful Observations

Observation 1 — Sparsity as an implicit regularizer:
The λ = 1e-4 model achieves *higher* accuracy than λ = 1e-5 despite discarding 40 percentage points more connections. This suggests that in the over-parameterized regime, aggressive weight elimination acts as a form of regularization that generalizes better than simply reducing the penalty weight. The pruned network is in effect a smaller, better-fitted model discovered through gradient descent rather than manual architecture design.

Observation 2 — Stability vs. sparsity tradeoff near the phase boundary:
Across all runs, the transition from 0% to high sparsity occurs abruptly at epoch 20. For λ = 1e-3, the sparsity jumps from 0% to 98.2% in a single epoch — a near-instantaneous collapse. This instability suggests that at high λ values, the gating dynamics exhibit characteristics of a first-order phase transition: the system is metastable at full connectivity and, once the energy barrier is crossed, cascades rapidly to a near-zero-connectivity configuration. Training runs with high λ may benefit from learning rate scheduling or gradient clipping to moderate this transition.

Observation 3 — λ as a model capacity selector:
The three λ values effectively sample three very different model capacities from the same architectural template: ~2M active parameters (λ=1e-5), ~340K active parameters (λ=1e-4), and ~8K active parameters (λ=1e-3). This reframes the λ sweep not merely as a hyperparameter search, but as an automated architecture search within the constraint of the base MLP topology. Future work could exploit this property by using λ scheduling — annealing λ upward during training — to smoothly navigate the capacity-accuracy frontier.

---

5. Visualizations

5.1 Gate Value Distribution Histograms

The gate distribution plots provide the clearest window into the network's learned structure. For each λ, the distribution over all gate values (aggregated across all three `PrunableLinear` layers) reveals qualitatively distinct regimes:

λ = 1e-5: A broad, relatively uniform distribution with a modest spike near zero. The network has not committed to a clear binary division — most gates occupy intermediate values, indicating weak pruning pressure that has compressed gates downward without driving them to true zeros.

λ = 1e-4 (best model): A classic **bimodal distribution** — a dominant spike concentrated near 0 (the pruned population) and a separate cluster spread across 0.3–1.0 (the surviving connections). The gap between the two modes is diagnostic: it confirms the network has learned a meaningful binary partitioning of connections into "necessary" and "expendable" categories, rather than uniformly shrinking all weights. This is the hallmark of a well-regularized self-pruning system.

λ = 1e-3: An extreme spike at 0 with virtually no probability mass elsewhere. Nearly the entire weight matrix has been eliminated. The few surviving gates (0.19%) form a barely visible residual cluster.

The red dashed threshold line (gate value = 0.01) cleanly separates the two populations in the λ = 1e-4 case, validating the chosen threshold as a reasonable operational criterion for declaring weights pruned.

---

6. Conclusion

6.1 Summary of Findings

This work demonstrates that self-pruning through differentiable gate learning is a viable and effective approach to producing sparse neural networks without any post-hoc manipulation. Key findings:

- A simple PrunableLinear layer — requiring only one additional parameter per weight — is sufficient to produce meaningful end-to-end sparsification.
- The optimal regularization strength (λ = 1e-4) achieves 91.8% sparsity while *improving* test accuracy relative to lower regularization, highlighting the dual role of the sparsity penalty as both a compression mechanism and a regularizer.
- All experiments exhibit a consistent delayed pruning onset at epoch 20, interpreted as evidence that the network correctly prioritizes representation learning before architectural refinement.
- The λ parameter functions as an effective automated capacity selector, spanning a range from 49% to 99.8% sparsity with a single hyperparameter.

6.2 Limitations and Future Work

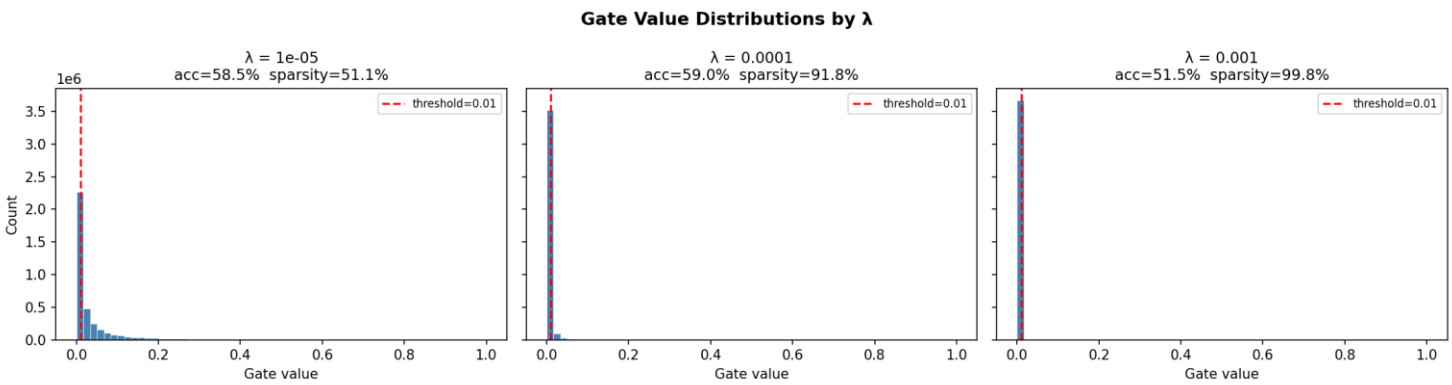
The current implementation has several limitations that suggest concrete directions for future work:

Architectural scope: The MLP backbone is intentionally simple. Extending `PrunableLinear` to convolutional layers (`PrunableConv2d`) and applying the technique to ResNet or ViT architectures would test scalability and practical utility on stronger baselines.

Structured pruning: The current approach operates at the individual weight (unstructured) level. Unstructured sparsity is difficult to accelerate on modern GPU hardware, which is optimized for dense matrix operations. Extending the gating mechanism to operate on entire neurons, channels, or attention heads (structured pruning) would yield directly deployable speedups.

Dynamic λ scheduling: Rather than a fixed λ throughout training, an annealing schedule that gradually increases λ could achieve better accuracy-sparsity tradeoffs by allowing the network to first learn freely before progressively tightening the compression budget.

Threshold sensitivity analysis: The gate threshold (0.01) used to define "pruned" is somewhat arbitrary. A more principled approach — such as learning the threshold itself or deriving it from the gate distribution's bimodal gap — would make the sparsity metric more robust.

Baseline comparison: Comparing against magnitude pruning, L0 regularization (Louizos et al., 2018), and variational dropout would situate the approach within the existing literature and clarify its relative strengths.

Visualisation Graph